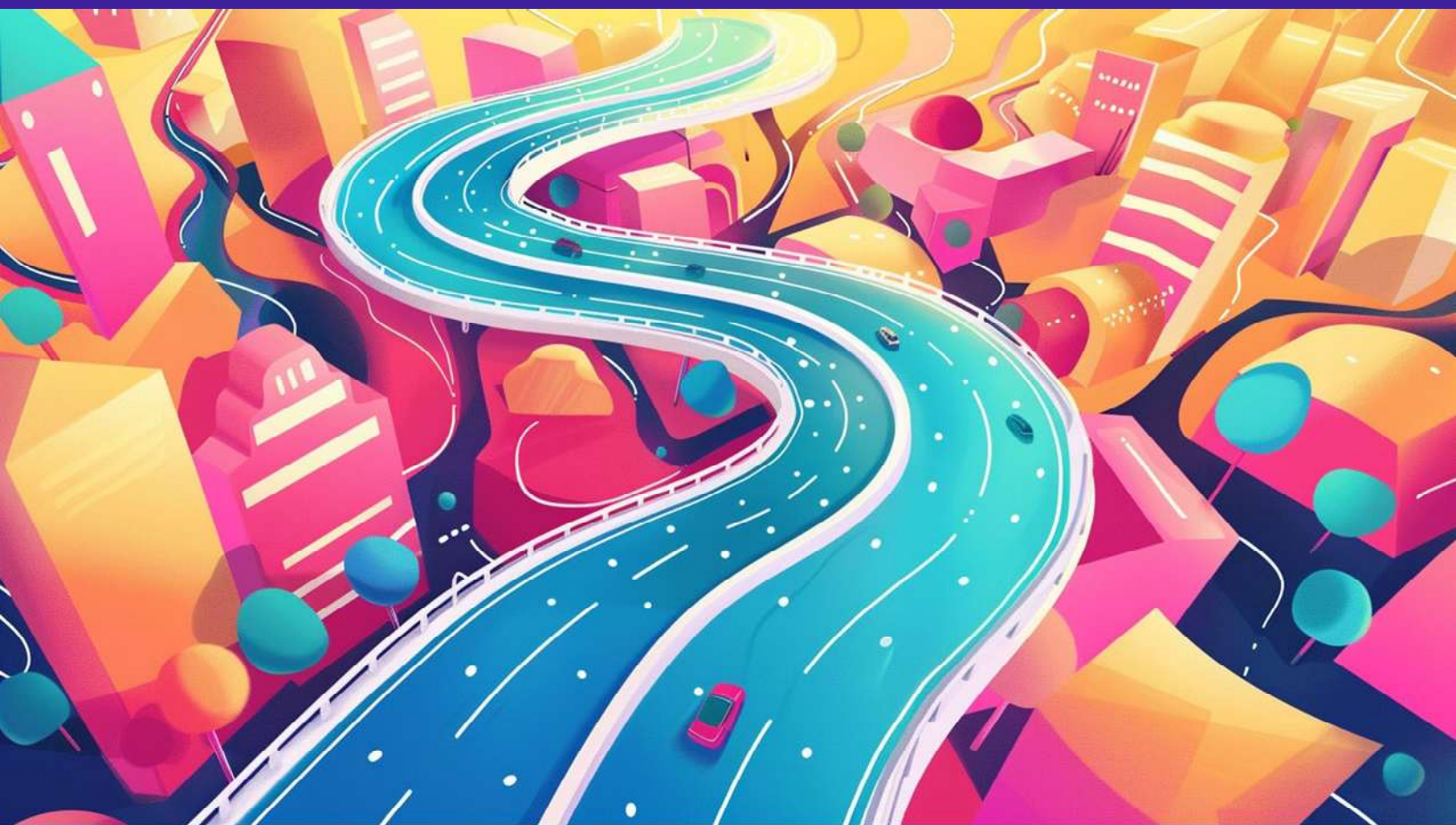


THE COMPLETE FRONT-ENDDEVELOPER ROADMAP



[lokanath-mohanty](#)

Journey to Frontend Mastery



Introduction

Frontend development involves building the user interface and experience of websites or web applications using web technologies like HTML, CSS, and JavaScript.

Who is this for?

- **Beginners** who want to land a frontend developer job.
- **Intermediate learners** wanting to fill knowledge gaps.

1-Year Learning Plan Overview

<u>Skill</u>	<u>Time Required</u>	<u>Learning Phase</u>
HTML	2 weeks	Beginner
CSS	1 month	Beginner
JavaScript	2 months	Beginner
Git	2 weeks	Beginner
TypeScript	3 weeks	Intermediate
React	2 months	Intermediate
SASS	2 weeks	Intermediate
Tailwind CSS	3 weeks	Intermediate
Automated Testing	1 month	Advanced
Next.js	1.5 months	Advanced
React Native	2 months (optional)	Advanced

Top Tips for Beginners

- **Start Small and Build Up** – Learn fundamentals before advanced frameworks.
- **Practice Consistently** – Code daily to retain concepts.
- **Work on Projects** – Apply learning in real-world projects.
- **Ask for Help** – Use ChatGPT, Stack Overflow, or Discord communities.
- **Join Communities** – Follow Frontend Mentor, Hashnode, or Reddit.
- **Learn to Debug** – Understand error messages and use dev tools.
- **Use Rubber Duck Debugging** – Explain code out loud to catch bugs.
- **Read Documentation** – Official docs are the best resource.
- **Stay Updated** – Follow dev blogs, Twitter, or YouTube channels.
- **Be Patient & Persistent** – Learning to code takes time.
- **Embrace Change** – Web tech evolves fast—adapt continuously.
- **Take Breaks** – Rest improves clarity and productivity.

HTML – HyperText Markup Language

Duration: 1–2 weeks

Purpose: Structure the content of web pages.

Key Concepts:

Tags: <html>, <head>, <body>, <h1> to <h6>, <p>, <a>,

Forms: <form>, <input>, <label>, <button>

Semantic tags: <section>, <article>, <footer>, <nav>

Multimedia: <audio>, <video>, <source>

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>My Portfolio</title>
```

```
  </head>
```

```
  <body>
```

```
    <header>
```

```
      <h1>Welcome to My Website</h1>
```

```
    </header>
```

```
    <section>
```

```
      <p>This is a paragraph.</p>
```

```
      <a href="https://linkedin.com">My LinkedIn</a>
```

```
    </section>
```

```
  </body>
```

```
</html>
```

CSS – Cascading Style Sheets

Duration: 2–4 weeks

Purpose: Add styles (color, layout, responsiveness)

Key Concepts:

Selectors: .class, #id, element, :hover, ::before

Box Model: margin, padding, border

Flexbox & Grid for layout

Responsive Design: Media queries

Transitions, animations

Example:

```
body {
```

```
  font-family: Arial, sans-serif;
```

```
  background-color: #f5f5f5;
```

```
}
```

```
h1 {
```

```
  color: #333;
```

```
}
```

```
@media (max-width: 600px) {
```

```
  h1 {
```

```
    font-size: 20px;
```

```
}
}
```

HTML/CSS Project Ideas:

- **Portfolio Website**
- **Responsive Blog Layout**
- **Product Landing Page**

JavaScript

Duration: 6–8 weeks

Purpose: Adds interactivity (clicks, form validation, dynamic content)

Key Concepts:

Variables: let, const, var

Functions, Objects, Arrays, Loops

DOM Manipulation: document.getElementById()

Async JS: fetch, Promises, async/await

Event Handling: click, submit, keyup

Example:

```
document.getElementById("btn").addEventListener("click", function () {
  alert("Button clicked!");
});
```

JavaScript Project Ideas

- **To-Do App**
- **Weather App (API)**
- **Image Slider**
- **Quiz App**

Git + GitHub

Duration: 2 weeks

Purpose: Version control, code collaboration

Key Concepts:

git init, git clone, git status, git add, git commit

Branching: git branch, git checkout

Remote: git push, git pull, git fetch

Platforms: GitHub, GitLab

TypeScript

Duration: 2–3 weeks

Purpose: Typed JavaScript – fewer bugs, better editor support

Example:

```
function greet(name: string): void {
  console.log("Hello, " + name);
}
```

```
greet("Loknath");
```

React.js

Duration: 6–8 weeks

Purpose: Build dynamic UIs with components

Key Concepts:

Components, Props, State, JSX

Hooks: useState, useEffect

Event handling, forms

Routing: React Router

Data fetching: axios, fetch

Example:

```
import { useState } from 'react';
```

```
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

React Project Ideas

- **Todo List**
- **Weather App**
- **Recipe Finder**
- **Expense Tracker**
- **E-Commerce Frontend**

SASS

Duration: 1–2 weeks

Purpose: CSS with variables, nesting, functions

Example:

```
$primary-color: #3498db;
```

```
button {
  background-color: $primary-color;
```

```
&:hover {
  background-color: darken($primary-color, 10%);
}
}
```

Tailwind CSS

Duration: 2–3 weeks

Purpose: Utility-first CSS framework

Example:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-700">
  Click Me
</button>
```

Automated Testing (Jest/Vitest)

Duration: 3–4 weeks

Purpose: Ensures your code works correctly

Example (Jest):

```
test("adds 2 + 2", () => {
  expect(2 + 2).toBe(4);
});
```

Next.js

Duration: 4–6 weeks

Purpose: React framework with server-side rendering and SEO

Key Concepts:

Pages and routing

Data fetching: `getStaticProps`, `getServerSideProps`

API Routes

Auth: `NextAuth.js`

Styling: Tailwind, CSS Modules

Next.js Project Ideas

- Personal Blog
- E-commerce Store
- Event Booking
- Recipe Sharing Platform

React Native (Optional)

Duration: 6–8 weeks

Purpose: Build mobile apps with React

Concepts:

Core Components: View, Text, Image, ScrollView

Navigation: react-navigation

Native modules: AsyncStorage, Push Notifications

React Native Project Ideas

- **Fitness Tracker**
- **Expense Tracker**
- **News App**
- **Shopping Cart App**

Final Notes

- By following this roadmap and building projects at every stage, you will:
- Master frontend fundamentals
- Create a strong portfolio
- Be ready to apply for junior frontend developer roles

THANK YOU